

```
"""
```

```
iPhone Deal Scout Bot
```

```
=====
```

```
Scrape Vinted & LeBonCoin for iPhone deals, compare prices,  
and send alerts via email or push notification.
```

```
Usage:
```

```
    pip install -r requirements.txt  
    python scraper_bot.py --config config.json
```

```
Ethics & Best Practices:
```

- Respects robots.txt
- Random delays between requests (3-10s)
- User-Agent rotation
- Rate-limiting built-in
- No login scraping / credential abuse

```
"""
```

```
import asyncio  
import json  
import logging  
import random  
import re  
import smtplib  
import time  
from dataclasses import dataclass, field, asdict  
from datetime import datetime  
from email.mime.multipart import MIMEMultipart  
from email.mime.text import MIMEText  
from pathlib import Path  
from typing import Optional  
from urllib.parse import urlencode, urljoin  
from urllib.robotparser import RobotFileParser
```

```
import requests  
from bs4 import BeautifulSoup
```

```
# ——— Logging —————
```

```
logging.basicConfig(  
    level=logging.INFO,  
    format="%(asctime)s [%(levelname)s] %(name)s - %(message)s",  
    handlers=[  
        logging.StreamHandler(),
```

```

        logging.FileHandler("deal_bot.log", encoding="utf-8"),
    ],
)
log = logging.getLogger("DealBot")

```

— Data Models —————

```

@dataclass
class WatchRule:
    """A single monitoring rule defined by the user."""
    model: str # e.g. "iPhone 14 Pro"
    max_price: float # alert if price <= this
    min_storage: Optional[int] = None # GB, optional
    condition: str = "any" # "new" | "like_new" | "good" | "any"
    platforms: list = field(default_factory=lambda: ["vinted", "leboncoin"])
    keywords_exclude: list = field(default_factory=list) # block these keywords

```

```

@dataclass
class Listing:
    """A single marketplace listing."""
    platform: str
    title: str
    price: float
    url: str
    image_url: str = ""
    condition: str = "unknown"
    location: str = ""
    published_at: str = ""
    listing_id: str = ""

    @property
    def is_valid(self) -> bool:
        return self.price > 0 and bool(self.url)

```

```

@dataclass
class AlertConfig:
    """Email / notification settings."""
    email_enabled: bool = False
    smtp_host: str = "smtp.gmail.com"
    smtp_port: int = 587
    smtp_user: str = ""
    smtp_password: str = ""
    recipient_email: str = ""
    # Pushover (mobile push)

```

```

    pushover_enabled: bool = False
    pushover_user_key: str = ""
    pushover_api_token: str = ""

# — Robots.txt Checker —————

_robots_cache: dict[str, RobotFileParser] = {}

def can_fetch(base_url: str, path: str, user_agent: str = "") -> bool:
    """Check robots.txt before scraping a URL."""
    if base_url not in _robots_cache:
        rp = RobotFileParser()
        rp.set_url(urljoin(base_url, "/robots.txt"))
        try:
            rp.read()
        except Exception:
            return True # if unreadable, assume allowed
        _robots_cache[base_url] = rp
    return _robots_cache[base_url].can_fetch(user_agent, path)

# — HTTP Session —————

USER_AGENTS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 14_4) AppleWebKit/605.1.15 "
    "(KHTML, like Gecko) Version/17.4 Safari/605.1.15",
    "Mozilla/5.0 (X11; Linux x86_64; rv:125.0) Gecko/20100101 Firefox/125.0",
]

def _make_session() -> requests.Session:
    s = requests.Session()
    s.headers.update({
        "User-Agent": random.choice(USER_AGENTS),
        "Accept-Language": "fr-FR,fr;q=0.9,en;q=0.8",
        "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0."
    })
    return s

def polite_get(session: requests.Session, url: str, **kwargs) -> Optional[request
    """GET with random delay and error handling."""
    delay = random.uniform(3, 10)
    log.debug(f"Waiting {delay:.1f}s before fetching {url}")
    time.sleep(delay)

```

```

try:
    resp = session.get(url, timeout=15, **kwargs)
    resp.raise_for_status()
    return resp
except requests.RequestException as e:
    log.warning(f"Request failed for {url}: {e}")
    return None

# — Platform Scrapers —————

class VintedScraper:
    """
    Scrapes Vinted public search results.
    Uses Vinted's public catalog endpoint (no login required).
    """
    BASE = "https://www.vinted.fr"
    API = "https://www.vinted.fr/api/v2/catalog/items"

    def search(self, session: requests.Session, rule: WatchRule) -> list[Listing]:
        if not can_fetch(self.BASE, "/api/v2/catalog/items"):
            log.warning("Vinted: robots.txt disallows scraping - skipping.")
            return []

        params = {
            "search_text": rule.model,
            "price_to": rule.max_price,
            "catalog_ids": "2,16", # Electronics > Téléphones portables
            "order": "newest_first",
            "per_page": 40,
        }

        if rule.condition != "any":
            status_map = {"new": "6", "like_new": "1", "good": "2"}
            if rule.condition in status_map:
                params["status_ids"] = status_map[rule.condition]

        url = f"{self.API}?{urlencode(params)}"
        resp = polite_get(session, url, headers={"Accept": "application/json"})
        if resp is None:
            return []

        try:
            data = resp.json()
        except ValueError:
            log.error("Vinted: failed to parse JSON response")
            return []

```

```

listings = []
for item in data.get("items", []):
    price_raw = item.get("price", {})
    price = float(price_raw.get("amount", 0)) if isinstance(price_raw, dict) else 0
    listing = Listing(
        platform="Vinted",
        listing_id=str(item.get("id", "")),
        title=item.get("title", ""),
        price=price,
        url=f"{self.BASE}/items/{item.get('id')}",
        image_url=(item.get("photos") or [{}])[0].get("url", ""),
        condition=item.get("status", "unknown"),
        location=item.get("city", ""),
        published_at=item.get("updated_at_ts", ""),
    )
    if listing.is_valid:
        listings.append(listing)

log.info(f"Vinted: found {len(listings)} results for '{rule.model}'")
return listings

```

```

class LeBonCoinScraper:

```

```

    """

```

```

    Scrapes LeBonCoin search results via public HTML.

```

```

    Category 43 = Téléphones (Electronics).

```

```

    """

```

```

    BASE = "https://www.leboncoin.fr"

```

```

    SEARCH = "https://www.leboncoin.fr/recherche"

```

```

    def search(self, session: requests.Session, rule: WatchRule) -> list[Listing]

```

```

        if not can_fetch(self.BASE, "/recherche"):

```

```

            log.warning("LeBonCoin: robots.txt disallows - skipping.")

```

```

            return []

```

```

        params = {

```

```

            "text": rule.model,

```

```

            "category": "43",          # Téléphones portables

```

```

            "price": f"0-{int(rule.max_price)}",

```

```

            "sort": "time",

```

```

            "order": "desc",

```

```

        }

```

```

        url = f"{self.SEARCH}?{urlencode(params)}"

```

```

        resp = polite_get(session, url)

```

```

        if resp is None:

```

```

            return []

```

```

soup = BeautifulSoup(resp.text, "html.parser")

# LeBonCoin injects __NEXT_DATA__ JSON into the page
script = soup.find("script", {"id": "__NEXT_DATA__"})
if not script:
    log.warning("LeBonCoin: could not find __NEXT_DATA__ - layout may hav
    return []

try:
    page_data = json.loads(script.string)
    ads = (
        page_data
        .get("props", {})
        .get("pageProps", {})
        .get("searchData", {})
        .get("ads", [])
    )
except (ValueError, AttributeError):
    log.error("LeBonCoin: failed parsing __NEXT_DATA__")
    return []

listings = []
for ad in ads:
    price_raw = ad.get("price", [None])[0] if ad.get("price") else None
    price = float(price_raw) if price_raw else 0.0
    listing = Listing(
        platform="LeBonCoin",
        listing_id=str(ad.get("list_id", "")),
        title=ad.get("subject", ""),
        price=price,
        url=urljoin(self.BASE, ad.get("url", "")),
        image_url=(ad.get("images", {}).get("small_url") or ""),
        condition=ad.get("attributes", [{}])[0].get("value_label", "unkno
        location=ad.get("location", {}).get("city", ""),
        published_at=ad.get("first_publication_date", ""),
    )
    if listing.is_valid:
        listings.append(listing)

log.info(f"LeBonCoin: found {len(listings)} results for '{rule.model}'")
return listings

```

— Price Comparison & Deal Scoring —————

```

# Reference market prices (EUR) - update periodically
MARKET_PRICES: dict[str, float] = {

```

```

    "iphone 16 pro max": 1299,
    "iphone 16 pro":      1099,
    "iphone 16":          969,
    "iphone 15 pro max": 1099,
    "iphone 15 pro":      999,
    "iphone 15":          829,
    "iphone 14 pro max":  899,
    "iphone 14 pro":      799,
    "iphone 14":          629,
    "iphone 13 pro":      649,
    "iphone 13":          499,
    "iphone 12 pro":      449,
    "iphone 12":          349,
    "iphone se":          259,
}

def get_market_price(title: str) -> Optional[float]:
    title_lower = title.lower()
    for model, price in sorted(MARKET_PRICES.items(), key=lambda x: -len(x[0])):
        if model in title_lower:
            return price
    return None

def score_deal(listing: Listing) -> dict:
    """
    Returns a deal score 0-100 and a discount percentage.
    """
    market = get_market_price(listing.title)
    if market is None or market == 0:
        return {"score": 50, "discount_pct": 0, "market_price": None}
    discount_pct = (1 - listing.price / market) * 100
    # Score: 50 at 0% discount, 100 at 60%+ discount
    score = min(100, max(0, 50 + discount_pct * 0.833))
    return {
        "score": round(score),
        "discount_pct": round(discount_pct, 1),
        "market_price": market,
    }

def filter_listing(listing: Listing, rule: WatchRule) -> bool:
    """Return True if the listing matches the rule."""
    title_lower = listing.title.lower()
    # Must mention the model
    if rule.model.lower() not in title_lower:
        return False
    # Price ceiling

```

```

    if listing.price > rule.max_price:
        return False
    # Exclude blacklisted keywords
    for kw in rule.keywords_exclude:
        if kw.lower() in title_lower:
            return False
    # Storage filter (e.g. "128 GB")
    if rule.min_storage:
        match = re.search(r"(\d+)\s*gb", title_lower)
        if match and int(match.group(1)) < rule.min_storage:
            return False
    return True

# — Alerting —————

_alerted_ids: set[str] = set()    # avoid duplicate alerts

def _build_email_html(matches: list[dict]) -> str:
    rows = ""
    for m in matches:
        l: Listing = m["listing"]
        d = m["deal"]
        discount_badge = (
            f'<span style="color:#22c55e;font-weight:bold;">-{d["discount_pct"]}%
            if d["discount_pct"] > 0 else ""
        )
        rows += f"""
<tr>
    <td style="padding:12px;border-bottom:1px solid #1e293b;">
        <a href="{l.url}" style="color:#38bdf8;text-decoration:none;font-weig
        <small style="color:#94a3b8;">{l.platform} · {l.location} · {l.publis
    </td>
    <td style="padding:12px;border-bottom:1px solid #1e293b;text-align:right
        <strong style="font-size:1.1em;color:#f1f5f9;">{l.price:.0f} €</stron
        {discount_badge}
    </td>
    <td style="padding:12px;border-bottom:1px solid #1e293b;text-align:cent
        <span style="background:#0f172a;color:#38bdf8;padding:4px 10px;border
        Score {d["score"]}/100
        </span>
    </td>
</tr>"""

    return f"""<!DOCTYPE html>

<html><body style="background:#0f172a;color:#f1f5f9;font-family:'Segoe UI',sans-s
<div style="max-width:680px;margin:32px auto;background:#1e293b;border-radius:12p

```



```

<div style="background:linear-gradient(135deg,#0ea5e9,#6366f1);padding:28px 32px">
  <h1 style="margin:0;font-size:1.4em;letter-spacing:-0.5px;">🔥 iPhone Deal Sc
  <p style="margin:6px 0 0 0;opacity:0.85;font-size:0.9em;">{len(matches)} bonne(
</div>
<table style="width:100%;border-collapse:collapse;">
  <thead>
    <tr style="background:#0f172a;">
      <th style="padding:10px 12px;text-align:left;color:#64748b;font-size:0.78em;">
      <th style="padding:10px 12px;text-align:right;color:#64748b;font-size:0.78em;">
      <th style="padding:10px 12px;text-align:center;color:#64748b;font-size:0.78em;">
    </tr>
  </thead>
  <tbody>{rows}</tbody>
</table>
<p style="padding:20px 32px;margin:0;color:#475569;font-size:0.8em;">
  Alerte générée le {datetime.now().strftime("%d/%m/%Y à %H:%M")} · iPhone Deal
</p>
</div>
</body></html>""

```

```

def send_email_alert(cfg: AlertConfig, matches: list[dict]) -> bool:
    if not cfg.email_enabled or not matches:
        return False
    try:
        msg = MIMEMultipart("alternative")
        msg["Subject"] = f"🔥 {len(matches)} bonne(s) affaire(s) iPhone détectée("
        msg["From"] = cfg.smtp_user
        msg["To"] = cfg.recipient_email
        msg.attach(MIMEText(_build_email_html(matches), "html", "utf-8"))

        with smtplib.SMTP(cfg.smtp_host, cfg.smtp_port) as server:
            server.starttls()
            server.login(cfg.smtp_user, cfg.smtp_password)
            server.sendmail(cfg.smtp_user, cfg.recipient_email, msg.as_string())
            log.info(f"Email sent to {cfg.recipient_email}")
        return True
    except Exception as e:
        log.error(f"Email failed: {e}")
        return False

```

```

def send_pushover_alert(cfg: AlertConfig, matches: list[dict]) -> bool:
    """Send mobile push notification via Pushover (free tier supported)."""
    if not cfg.pushover_enabled or not matches:
        return False
    top = matches[0]["listing"]

```

```

deal = matches[0]["deal"]
message = (
    f"🔥 {top.title}\n"
    f"💰 {top.price:.0f} € "
    + (f"(-{deal['discount_pct']})%" if deal["discount_pct"] > 0 else "")
    + f"\n📍 {top.platform} · {top.location}\n{top.url}"
)
payload = {
    "token": cfg.pushover_api_token,
    "user": cfg.pushover_user_key,
    "title": f"iPhone Deal Scout - {len(matches)} affaire(s)",
    "message": message,
    "url": top.url,
    "url_title": "Voir l'annonce",
    "priority": 0,
}
try:
    r = requests.post("https://api.pushover.net/1/messages.json", data=payload)
    r.raise_for_status()
    log.info("Pushover notification sent")
    return True
except Exception as e:
    log.error(f"Pushover failed: {e}")
    return False

# — Main Bot Loop —————

class DealBot:
    def __init__(self, rules: list[WatchRule], alert_cfg: AlertConfig,
                 interval_minutes: int = 15, min_score: int = 60):
        self.rules = rules
        self.alert_cfg = alert_cfg
        self.interval = interval_minutes * 60
        self.min_score = min_score
        self.scrapers = {
            "vinted": VintedScraper(),
            "leboncoin": LeBonCoinScraper(),
        }
        self.session = _make_session()

    def _scan_rule(self, rule: WatchRule) -> list[dict]:
        found_matches = []
        for platform in rule.platforms:
            scraper = self.scrapers.get(platform)
            if scraper is None:
                log.warning(f"Unknown platform: {platform}")

```

```

        continue
    try:
        listings = scraper.search(self.session, rule)
    except Exception as e:
        log.error(f"{platform} scraper error: {e}")
        continue

    for listing in listings:
        uid = f"{listing.platform}_{listing.listing_id}"
        if uid in _alerted_ids:
            continue
        if not filter_listing(listing, rule):
            continue
        deal = score_deal(listing)
        if deal["score"] < self.min_score:
            continue
        found_matches.append({"listing": listing, "deal": deal, "rule": r
        _alerted_ids.add(uid)

    # Sort best deals first
    found_matches.sort(key=lambda x: -x["deal"]["score"])
    return found_matches

def run_once(self) -> list[dict]:
    all_matches = []
    for rule in self.rules:
        log.info(f"Scanning: {rule.model} (max {rule.max_price}€)")
        matches = self._scan_rule(rule)
        all_matches.extend(matches)
        log.info(f" → {len(matches)} new deal(s) found")

    if all_matches:
        send_email_alert(self.alert_cfg, all_matches)
        send_pushover_alert(self.alert_cfg, all_matches)

    return all_matches

def run_forever(self):
    log.info(f"Bot started – scanning every {self.interval // 60} min "
            f"for {len(self.rules)} rule(s).")
    while True:
        try:
            self.run_once()
        except Exception as e:
            log.error(f"Unexpected error in main loop: {e}")
        log.info(f"Next scan in {self.interval // 60} min...")
        time.sleep(self.interval)

```

```

# — Config Loader —————

def load_config(path: str) -> tuple[list[WatchRule], AlertConfig, dict]:
    with open(path, encoding="utf-8") as f:
        cfg = json.load(f)

        rules = [WatchRule(**r) for r in cfg.get("rules", [])]
        alert_cfg = AlertConfig(**cfg.get("alerts", {}))
        bot_cfg = cfg.get("bot", {"interval_minutes": 15, "min_score": 60})
        return rules, alert_cfg, bot_cfg

# — Entry Point —————

if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="iPhone Deal Scout Bot")
    parser.add_argument("--config", default="config.json", help="Path to config f
    parser.add_argument("--once", action="store_true", help="Run a single scan th
    args = parser.parse_args()

    rules, alert_cfg, bot_cfg = load_config(args.config)
    bot = DealBot(
        rules=rules,
        alert_cfg=alert_cfg,
        interval_minutes=bot_cfg.get("interval_minutes", 15),
        min_score=bot_cfg.get("min_score", 60),
    )

    if args.once:
        results = bot.run_once()
        print(f"\n✅ Scan terminé - {len(results)} affaire(s) trouvée(s).")
        for m in results:
            l = m["listing"]
            d = m["deal"]
            print(f"  [{l.platform}] {l.title} - {l.price}€ (score {d['score']})/1
    else:
        bot.run_forever()

```